## Problem 1

Consider the following algorithm SILLYSORT and its subroutine SMERGE. Aptly named, SILLYSORT is a `MergeSort`-like algorithm that sorts the given array $R[1..n]$ of $n$ integers. For all parts, assume that $n$ is a power of two and $R$ contains distinct integers (no duplicates).

```
SILLYSORT(R[1..n]):
  if n > 1:
      SILLYSORT(R[1..n/2])
      SILLYSORT(R[n/2+1..n])
      SMERGE(R[1..n])
```

```
SMERGE(R[1..n]):
  if n = 2:
      if R[1] > R[2]:
          swap R[1] ⟷ R[2]
  else:
      for i ← 1 to n/4:
          swap R[i + n/4] ⟷ R[i + n/2]      ⟨⟨swap middle fourths⟩⟩
      SMERGE(R[1..n/2])      ⟨⟨leftmost half⟩⟩
      SMERGE(R[n/2+1..n])    ⟨⟨rightmost half⟩⟩
      SMERGE(R[n/4+1..3n/4]) ⟨⟨center half⟩⟩
```

Note: The notation $R[i..j]$ is a pointer to the subarray of an array $R$ from indices $i$ to $j$ (inclusive). Any operations on a subarray are in-place and thus occur on the original array that contains it. (This is the same as for the description of MERGESORT and QUICKSORT from our textbook/lecture.)

(a) Use induction to prove that `Smerge`$(R[1..n])$ sorts the given array $R$ in-place, assuming that $R[1..n/2]$ and $R[n/2+1...n]$ are sorted. *[Hint: It may be useful to first follow the smallest $n/4$ and largest $n/4$ elements in $R$ before/between/after the initial for-loop and recursive calls.]*

(b) State a recurrence that describes the runtime of `Smerge`, then solve it using the recursion tree method.

(c) State a recurrence that describes the runtime of `SillySort`, then solve it using the recursion tree method.

*Additional ungraded exercise–do not submit:* Prove that SILLYSORT$(R[1..n])$ indeed sorts $R$ using induction, assuming the correctness of `Smerge` from part (a).

**Solution:**

(a) **Proof:** We prove the claim by induction on the size of the input array. Let $A[1..n]$ be an arbitrary array of distinct integers, where $n$ is a power of two and $A[1..n/2]$ and $A[n/2+1..n]$ are sorted. Assume that SMERGE sorts any given array smaller than $A$ whose length is a power of two and its first and second halves are sorted.

- *Base cases.* If $n \leq 1$, then $A$ is empty or contains a single element and is thus trivially sorted. If $n = 2$, then $A$ consists of two elements, and the first if-case correctly swaps its elements into sorted order as necessary.
- *Inductive case.* If $n > 2$, then $n \geq 4$. For conciseness, let $A_i := A[(in/4)..(i+1)n/4]$ for each $i \in \{0,1,2,3\}$; that is, we partition $A$ into four contiguous subarrays of equal size $n/4$, $A_0, A_1, A_2, A_3$, where $A_0$ is the first quarter of $A$, and so on. Since $A[1..n/2]$ and $A[n/2+1..n]$ are sorted, each of the $A_i$'s are sorted from the start. Furthermore, all

*Problem 1 continued on next page...*

elements of $A_0$ are smaller than those in $A_1$, and all elements of $A_2$ are smaller than those in $A_3$. Thus, any of the smallest $n/4$ elements of $A$ must lie in either $A_0$ or $A_2$. By a similar argument, the largest $n/4$ elements of $A$ must lie in either $A_1$ or $A_3$.

The for-loop swaps the elements of $A_1$ and $A_2$ while maintaining their relative ordering. After the swap, $A_1$ and $A_2$ remain sorted, and now the smallest (resp., largest) $n/4$ elements of $A$ lie in $A_0 \cup A_1$ (resp., $A_2 \cup A_3$). The induction hypothesis (IH) implies that $A[0..m-1] = A_0 \cup A_1$ is sorted by the first recursive call. Since this subarray contains the $n/4$ smallest elements of $A$, those elements are now in their correct, sorted positions in $A$. Similarly, the IH implies that $A[n/2+1..n] = A_2 \cup A_3$ is sorted by the second recursive call. Since this subarray contains the $n/4$ largest elements of $A$, those elements are now in their correct, sorted positions in $A$. Thus, after the first two recursive calls, $A_0$ and $A_3$ contain the smallest and largest $n/4$ elements of $A$ in correct order, and $A_2$ and $A_3$ are sorted. $A[n/4..3n/4] = A_1 \cup A_2$ contains the remaining $n/2$ elements to be placed correctly in $A$, which is sorted by the third and final recursive call by the IH. Therefore, $A$ is sorted when the algorithm terminates.

Our cases are exhaustive, and hence we conclude that the algorithm is correct by induction. □

(b) SMERGE has the following recurrence relation:

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

This is because each recursive call halves the size of the input, and there are 3 recursive calls. The FOR-LOOP is performed in each recursive call and operates on a fourth of the input, so it performs $O(n)$ work. Solving the recurrence relation via the recursion tree method, at level $i$, there is

$$n \cdot \left(\frac{3}{2}\right)^i$$

work. There are $\log_2 n$ levels, so the total amount of work is expressed as:

$$\begin{aligned}
&\sum_{i=0}^{\log_2 n} n \cdot \left(\frac{3}{2}\right)^i \\
&= n \cdot \sum_{i=0}^{\log_2 n} \left(\frac{3}{2}\right)^i \\
&= n \cdot O\left(\left(\frac{3}{2}\right)^{\log_2 n}\right) \\
&= n \cdot O(n^{\log_2(3/2)}) \\
&= O(n^{\log_2(3/2)+1}) \\
&= O(n^{\log_2 3})
\end{aligned}$$

We conclude that the runtime of SMERGE to be $O(n^{\log_2 3})$.

(c) SILLYSORT has the following recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + S(n)$$

*Problem 1 continued on next page...*

where $S(n) = O(n^{\log_2 3})$ is the runtime of SMERGE from (b). This is because each recursive call halves the size of the input, and there are 2 recursive calls. Solving the recurrence relation via the recursion tree method, at level $i$, the amount of work done is

$$2^i \cdot \left(\frac{n}{2^i}\right)^{\log_2 3}.$$

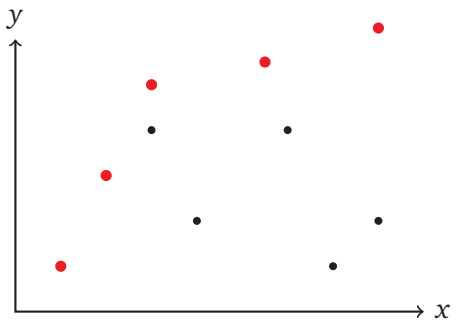
There $\log_2(n)$ levels, so the total amount of work is expressed as:

$$\begin{aligned}&\sum_{i=0}^{\log_2 n} 2^i \cdot \left(\frac{n}{2^i}\right)^{\log_2 3} \\ =&n^{\log_2 3} \cdot \sum_{i=0}^{\log_2 n} \left(\frac{2}{3}\right)^i\end{aligned}$$

The sum is a decreasing geometric sum, so it is dominated by the largest term, which is the term with index $i = 0$, $(2/3)^0 = 1$. It follows that the entire expression above is bounded above by $O(n^{\log_2 3})$. Therefore, the runtime is $O(n^{\log_2 3})$.

## Problem 2

Let $S = \{p_i = (x_i, y_i) : 1 \leq i \leq n\}$ be a set of $n$ points on a 2D plane. We say that a point $p_i$ is *northwest* (NW) of $p_j$ if $x_i \leq x_j$ and $y_i \geq y_j$, treating the $(-x)$-direction (resp., $(+y)$-direction) as west (resp., north). A point $p_i$ is a *northwesterly* point of $S$ if there are no other points in $S$ NW of $p_i$. In the example below, the larger red points are the northwesterly points of $S$.




Describe an algorithm that returns the northwesterly points of $S$ in $O(n \log n)$ time. Prove the correctness of your algorithm using induction, then state and justify its runtime using the recursion tree method. *[Hint: Sort then divide S into two sets based on the x-coordinates of points, and "conquer." What is the appropriate merge/combine step to obtain the overall solution?]*

**Solution:**
**Algorithm:** We first sort the array $S$ in increasing order of $x$-coordinate. If two points have the same $x$-coordinate, we place the one with higher $y$-coordinate first.

1. Our algorithm $FindNW(S[1..n])$ takes in an array of points, sorted using the above procedure, and will output an array consisting of its northwesterly points.

2. If $n = 1$, return $S$, composed of a single point. Otherwise, we (recursively) find the northwesterly points of the points left and right of the median $x$-coordinate, respectively. Specifically, we set $m := \lfloor n/2 \rfloor$, $L := S[1..m]$ and $R := S[m+1..n]$. Then we compute

$$NW_L := FindNW(L) \text{ and } NW_R := FindNW(R).$$

3. Then we compute

$$y^* = \max_{(x,y) \in L} y,$$

the maximum $y$-coordinate among of the left half of the points, $L$, by iterating through them.

4. Next, we find the subset of points $NW_R' \subseteq NW_R$ that have $y$-coordinate **strictly** greater than $y^*$. This is achieved by iterating through $NW_R$.

5. Finally we output $NW_L \cup NW_R'$ as the northwesterly points in $S$.

**Correctness:** Let $S[1..n]$ be an arbitrary set of points, sorted as described above. Assume that $FindNW$ returns the set of all northwesterly points on inputs smaller than $S$.

- *Base Case:* When $n = 1$, the algorithm simply returns $S$, which is indeed northwesterly as no point is to the northwest of it.

*Problem 2 continued on next page...*

- *Inductive Case:* Consider a sorted input of length $n \geq 2$. By the induction hypothesis, $FindNW(S[1...m])$, $FindNW(S[m+1...n])$ correctly compute the northwesterly points $NW_L, NW_R$ in the left and right halves, as their input is sorted. Consider the following four cases for a point $p \in S$:

  - $p \in (L \setminus NW_L) \cup (R \setminus NW_R)$. If $p \in L \setminus NW_L$ (resp., $p \in R \setminus NW_R$) then there is a point $q \in L$ (resp., $q \in R$) that is northwest of $p$, due to the correctness of $NW_L, NW_R$. $q \in S$ so $p$ is not northwesterly in $S$.
  - $p \in NW_L$. We claim $p$ is northwesterly in $S$. Indeed, by the correctness of $NW_L$ there is no point in $L$ that is northwest of $p$, and all points in $R$ either have strictly greater $x$-coordinate or strictly smaller $y$-coordinate (due to how we break ties in the initial sorting). So $p$ is northwesterly in $S$.
  - $p \in NW'_R$. We claim $p$ is northwesterly in $S$. Indeed, there is no point in $R$ that is northwest of it by the correctness of $NW_R$, and all points in $L$ have strictly smaller $y$-coordinate than that of $p$ (since it is in $NW'_R$). So $p$ is northwesterly in $S$.
  - $p \in NW \setminus NW'_R$. Then $p$ is not northwesterly, as follows. Let $(q_x, q_y)$ be the point with greatest $y$-coordinate in $S[1...m]$, so $q_y = y^*$, and let $(x_p, y_p) = p$. Then we have $x_q \leq x_p$ (due to sorting) and $y_q \geq y_p$ (by definition of $NW'_R$), so $q$ is northwest of $p$.

  Any point $p \in S$ satisfies exactly one of the four cases above. Thus, we have established that $NW_L \cup NW'_R$ is the set of all northwesterly points for $S$, which is returned by the algorithm.

**Runtime Analysis:** We obtain the following recurrence for the runtime $T(n)$ of $FindNW(S[1...n])$.

$$T(n) = 2T(n/2) + O(n).$$

Since we first recurse on two subproblems with half the input size, we have $2T(n/2)$. Computing $y^L_{max}$ from $L$ and $NW'_R$ from $NW_R$ take $O(n)$ time as they can each be done in a single for-loop over at most $n$ elements. The solution to this recursion is $T(n) = O(n \log n)$, either by observing that this is the same recurrence as mergesort, or by noting that the recursion tree has depth $O(\log n)$ and has total work $O(n)$ at each level. Since the initial sorting can be done in $O(n \log n)$ time, the total running time is $O(n \log n)$.

## Problem 3

An array $A[1..n]$ of $n$ distinct integers (no duplicates) is considered *twirled* if it consists of two non-empty contiguous sequences, $L$ and $R$, that are sorted in increasing order and all elements in $L$ are greater than those in $R$. In other words, if $R$ instead appeared left of $L$, the entire resulting sequence would be sorted in increasing order.

For example, in the following twirled array, the left subsequence is underlined and has length 6, the right subsequence is overlined and has length 4, and indeed all elements in the left subsequence are greater than those in the right subsequence:

$$[\underline{12,14,19,23,25,27},\overline{2,3,5,7}]$$

Describe an $O(\log n)$-time algorithm to return the length of the left subsequence of a given twirled array $A$. Prove its correctness using induction, then state and justify its runtime using the recursion tree method.

*[Hint: Consider a variant of binary search. Given an element $x$ of $A$, what information do you need to decide which of the left and right subsequences that $x$ belongs to?]*

**Solution:** The main idea is to determine whether the median element in the current subarray is the last element of the left subsequence. If yes, then we solve for the left subsequence's length using the median index and return it; in fact, the length is exactly the median index. Otherwise, we determine which side of the median element contains the last element of the left subsequence, then recursively search for it.

```
LeftLength(A[1..n]):
  if n = 2:
      return 1 ⟨⟨Both subseq. are non-empty, so each has size 1.⟩⟩
  m ← ⌊n/2⌋
  if A[m] > A[m+1]: ⟨⟨A[m] is last element of left subseq. in A⟩⟩
      return m
  else if A[1] ≤ A[m]: ⟨⟨A[m] lies in left subseq. in A⟩⟩
      return m + LeftLength(A[(m+1)..n])
  else: ⟨⟨A[m] lies in right subseq. in A⟩⟩
      return LeftLength(A[1..m])
```

**Correctness:** Let $A[1..n]$ be an arbitrary twirled array. Assume that $LeftLength$ returns the length of the left subsequence in any input twirled array smaller than $A$.

- *Base Case:* When $n = 2$, the left and right subsequences of $A$ each have length 1 (since they are non-empty), and 1 is returned.
- *Inductive Case:* The only element in a twirled array greater than its next (to the right) is the last in the left subsequence. The if-condition after setting $m$ checks if $A[m]$ is this element of $A$, and if yes, then $m$ is correctly returned. Otherwise, there are two cases for $A[m]$. Both subsequences of $A$ are sorted in increasing order and all elements of the right subsequence of $A$ are less than every element, including $A[1]$. Thus, if $A[1] \le A[m]$ then $A[m]$ is in the left subsequence of $A$, otherwise it is in the right subsequence of $A$. Let $j$ be the length of the left subsequence in $A$.
  - If $A[m]$ is in the left subsequence, then $j > m$ since $A[m]$ is not the last element of the left subsequence. It follows that $A[m+1..n]$ is twirled with left subsequence $A[m+1..j]$ and right subsequence $A[j+1..n]$. By the IH, the call LeftLength$(A[m+1..n])$ correctly returns the length of the left subsequence in $A[m+1..n]$, $j-m$, so $m$ plus the returned length is the desired length, which is returned.

– Otherwise $A[m]$ is in the right subsequence and $j < m$. Then $A[1..m]$ is indeed twirled with left subsequence $A[1..j]$ and right subsequence $A[j+1..m]$. By the IH, the call LEFTLENGTH($A[1..j]$) correctly returns the length of the left subsequence in $A$, $j$.

**Runtime Analysis:** The algorithm performs at most one recursive call with input size roughly $n/2$ and the non-recursive work is $O(1)$. Hence the runtime of LEFTLENGTH on a twirled array of length $n$ is described by $T(n) = T(n) + 1$, which solved to $O(\log n)$ using the recursion tree method (or observing that the recurrence is the same as for binary search and other examples).

## Problem 4

In the following, all arrays/sequences are 0-indexed (as in Python). A *flippable list* is a data structure $f$ that stores a finite sequence $S_f$ of distinct integers and supports the following two operations:

- PEEK($i$): Return the object at index $i$ in $S_f$, where 0 is the first index.
- FLIP($i, j$): Reverse ("flip") the subsequence $S_f[i..j-1]$ if $i < j-1$ and do nothing otherwise. (NOTE: Notice the $j-1$ in the definition. This is to mimic Python's syntax: `x[i:j]` returns the sublist of list `x` from `i` to `j-1`.)

**Example**: If $S_f = [4,5,2,1,3]$, PEEK(1) returns 5 and FLIP(1,4) modifies $S_f$ so that it becomes $\left[4, \underline{1,2,5}, 3\right]$, where the underlined symbols are those in the affected subsequence.

For this problem, you will complete an implementation of a method similar to the PARTITION subroutine for QUICKSORT in Python 3. Download the class file `flippable_list.py` and your template file `flartition.py` on Canvas in the Files section. The file `flippable_list.py` includes the `FlippableList` class, which supports the PEEK and FLIP operations. For any Python list $S$, a FlippableList object representing $S$ is constructed and returned by calling FLIPPABLELIST($S$). ***You may not directly access the `_lst` attribute of the FlippableList objects in your submission, otherwise you will receive a grade of 0.*** However, you may find checking it to be useful while testing your code. See also the helper functions provided in the file.

`flartition.py` is the file in which you will complete the following task and cite any/all collaboration before uploading it to Gradescope under "HW1 Problem 4 (Code)". This problem has no written component.

Let $f$ be a flippable list, let $i, j$ be indices of $S_f$, and let $p$ be an integer (the "pivot" value). Design and implement a recursive method FLARTITION($f, i, j, p$) that modifies the subsequence $S_f[i..j-1]$, using only PEEK and FLIP operations on $f$, and returns an index $k$ where:

1. all elements in $S_f[i..k-1]$ are all *less than* $p$, and
2. all elements in $S_f[k..j]$ are all *at least* $p$.

In other words, the returned index $k$ is the first index of an element in the modified subsequence that is at least $p$; if all elements in the subsequence are less than $p$, then $k = j$. [1]

**Example**: For flippable list $f$ with sequence $S_f = [4,9,330,1,2,5,8,16]$, a valid value of $S_f$ after FLARTITION($f, 2, 7, 6$) is $\left[4, 9, \underline{2,1,5,330,8}, 16\right]$, where the underlined elements are those in the affected subsequence. The return value of the call is 5, which is the index of the first element (330) at least the pivot value of 6. The orderings of the elements less than/at least the pivot value 6 are arbitrary.

*[Hint: Split the problem into two halves, conquer, and then finish. Note that the median index of a subsequence $S_f[i..j-1]$ can be computed as $(i+j)/2$, rounded down.]*

**Solution:** We provide 0-indexed (Python-like) pseudocode for this problem.

[1] Note that this method differs from the PARTITION subroutine from QUICKSORT in the notes/lecture. In particular, the parameter $p$ need not be in $S_f[i..j-1]$, and elements equal to $p$ may end up anywhere after the elements smaller than $p$, not necessarily before those greater than $p$.

*Problem 4 continued on next page...*

```
FLARTITION(F[0..n−1], i, j, p):
  if j − i = 1:
      if PEEK(i) ≥ p:
          return i
      else:
          return i + 1
  else:
      m ← ⌊(j + i)/2⌋
      k1 ← FLARTITION(F, i, m, p)
      k2 ← FLARTITION(F, m, j, p)
      FLIP(F, k1, k2)
      return k1 + (k2 − m)
```

The runtime of the algorthm above is described by recurrence $T(n) = 2T(n/2) + n$. Here, $n$ is the length of the subarray to be flartitioned, which in terms of the input parameters is $j - i$. The $O(n)$ non-recursive work follows from the `flip(i,j)` operation, whose runtime is linear in the number of elements being flipped, $j - i$. This solves to $O(n \log n)$ using the recursion tree method (or recognizing it is the same as for mergesort and other examples we have seen in the class).